

HQ Server and Sync Layer, Audit

H-2 fixed 18 September; the rest remain open, see section 5.

Repositories: wairegimaina/hq_server (25,542 lines, 47 modules) and the sync/ client in cirquen-labs (16,982 lines) | Reviewed: 18 September 2026 Fifth in the review set. Extends A1_ENGINEERING_AUDIT.pdf, which covered CalSoft only.

1. Why this document exists

The first four documents treated certificate issuance as ending in the Django application. It does not. HQ allocates certificate numbers, flips sessions to approved, and owns the conflict resolution that decides which version of a calibration record survives. This audit covers the parts that bear on whether a calibration record arrives intact and whether a certificate number means anything.

One correction frames everything below. A design rule was stated during this review: **certificate numbers are generated on the server only**. That rule is correct and it is the safer design. The code honours it in one branch and violates it in the other, and section 3 shows that the violation has already caused a production incident with a name.

2. What HQ gets right

Stated first, because section 3 onward is critical.

Allocation is properly serialised. `certificate_normalizer.get_next_number` acquires `pg_advisory_xact_lock` before reading sequence state and holds it to commit, so two concurrent HQ workers cannot allocate the same number. The docstring explains why. This is correct, deliberate concurrency engineering.

The tests are aimed at what can corrupt data. 41 tests across certificate numbering, certificate delivery, soft-delete cascade, last-write-wins upsert, null propagation, API keys, client-registration idempotency and sync-backlog alerting. Test *density* is low against 25,542 lines, but the targets are the right ones. These are the paths where a bug silently destroys a record rather than raising.

Soft delete is genuinely soft. Rows are flagged via `active_status` / `pending_delete`; no hard DELETE of sessions exists in either repository. A deleted session keeps its certificate number, which is why gap reuse cannot recycle an issued number.

Foreign-key dependency resolution. `mirror.py` fetches missing parent records recursively and topologically sorts tables before mirroring, so a child row never arrives before its parent. This is the sort of thing usually learned the hard way, and it is handled.

Per-device API keys with hashing, issuance, revocation and rate limiting, plus an `audit_log` table on the HQ side.

The write model is sound. `db_for_write` is unconditionally local, with rows pushed to HQ afterwards. A field site is never blocked by HQ being unreachable. This is the right architecture for the environment.

3. Certificate numbers: the rule, the violation, and the incident

3.1 What the code does

Branch	Behaviour	Honours "server only"?
HQ unreachable	certificate_number = None, status approved_pending_certificate, a PendingCertificate row queued; HQ allocates on sync	Yes
HQ reachable	Calls CalibrationSession.generate_certificate_number() -> select_for_update on the local database -> local_max + 1	No

The online branch is gated by `is_hq_online()`, which is only an HTTP health probe against the sync API. It does not redirect the write, and `Equiper/db_router.py` returns "default" for every write with no branch or hint check. So when HQ is reachable, a field site allocates from **its own** maximum, into its own database, and pushes the result up afterwards.

HQ issues per-device API keys, so there are several field sites. Each holds its own local maximum. Two sites approving sessions at similar maxima allocate the same BNH-####; each succeeds locally because `unique=True` is enforced per-database; both then reach HQ.

3.2 This is not hypothetical

`sync/cert_conflict_guard.py` exists specifically to repair it, and says so:

```
Two machines calibrating offline can each mint the same next-in-sequence
BNH-NNNN certificate number for two different session rows before either
side has synced. Once both reach HQ, one of those rows becomes an "orphan":
a different UUID holding a certificate_number that HQ already considers
authoritative for a different row, the exact "BNH-0093 problem" that
helper_scripts/fix_cert_conflicts.py and helper_scripts/cert_checker.py
were written to diagnose and fix by hand.
```

Two hand-written diagnostic scripts and a named incident, **BNH-0093**, sit behind that mixin. The collision is a known production event, and the guard was built to stop someone having to notice it and run a script.

One detail of the docstring no longer matches the code: it attributes the collision to machines minting *offline*, but the offline branch assigns no number at all. The live source is the **online** branch allocating from a local maximum across multiple sites. The mechanism differs; the outcome is the one described.

3.3 Finding H-1, The repair rennumbers an already-issued certificate

This is the most serious finding in this document.

The guard's repair sequence, in one local transaction:

1. Clear the orphan's certificate_number to release the UNIQUE constraint
2. Restore HQ's authoritative number on the row HQ considers correct
3. Assign the orphan the next free, non-conflicting BNH-NNNN

Steps 1 and 2 are right: HQ is authoritative, and the correct row keeps the number. The allocation in step 3 is carefully done, the candidate is chosen before anything is mutated and checked against **both** databases.

The problem is what step 3 means outside the database. The orphan session was approved and already held a certificate number. If its PDF was generated, printed and filed, which is the entire purpose of approving a session, then after the guard runs, **the filed document and the database disagree about that certificate's number**, and nothing records that this happened except a log line. There is no superseded marker, no reissue record, no operator notification.

For an ISO/IEC 17025 assessor this is the traceability chain breaking silently. A certificate in a hospital file cannot be reconciled with the record, and the number printed on it may now belong to a different device.

The guard is not wrong to exist. The policy it implements is wrong: a certificate number bound to an issued document must never be reassigned. The correct repair is to void and reissue with an explicit superseded record, or, far better, to remove the collision at source per section 3.1.

3.4 Finding H-2, Two allocation algorithms remain reachable

Even with the guard in place, two algorithms write BNH-#### to the same column: HQ allocates lowest-gap-first under an advisory lock on the HQ database; CalSoft allocates strictly max+1 under select_for_update on the local database. Each is individually correct. Neither lock is visible to the other. The guard is a reconciliation layer over an avoidable divergence.

The stated rule, server-only allocation, removes it entirely. Two ways to implement the rule:

- **HQ allocates on request.** The online branch asks HQ for a number instead of computing one. Strictly correct, one authority, and the health probe already present becomes meaningful rather than decorative.
- **Treat online like offline.** Always queue a PendingCertificate and let HQ allocate on sync. Simplest, uses a path that already works, and removes the branch altogether at the cost of a short delay before a number exists.

The second is less code and fewer states. Either satisfies the rule; the local allocation path should then be deleted rather than left available.

4. Conflict resolution

4.1 Finding H-3, The last-write-wins guard cannot engage on client uploads

upload_processing._upsert_entity implements a careful LWW guard. On conflict it appends:

```
WHERE hq_row.source_updated_at IS NULL
OR EXCLUDED.source_updated_at IS NULL
OR EXCLUDED.source_updated_at >= hq_row.source_updated_at
```

so an older edit cannot clobber a newer stored value. It is documented, and tests/test_upsert_lww.py and tests/test_apply_change_simplified_lww.py verify it.

The guard is conditional on `_LWW_COLUMN` in `clean_data`, the incoming payload must carry `source_updated_at`. The evidence says client payloads cannot:

- migrations/2026_add_source_updated_at.sql adds and backfills the column on **HQ** tables (SET `source_updated_at = updated_at`).
- The column appears in **no** client model, no occurrence anywhere in CalSoft/, core/ or Equiper/.
- sync/mirror.py records the consequence directly: mirroring HQ rows *down* raised column `"source_updated_at"` of relation `"..."` does not exist for every affected row, which is why the client now drops unknown columns.
- Both LWW tests inject `source_updated_at` into the payload explicitly.

The payload construction has now been traced end to end, and it confirms the guard cannot engage. The chain:

- sync/sync_agent_3.py builds each upload event as `"data": row["row_data"]`.

- `row_data` comes from `to_jsonb(t.*)` (`sync/sync_agent_2.py:487` and `:643`, `sync/smart_delete.py:56` and `:372`), literally every column of the **local** table, serialised to JSON.
- The local table has no `source_updated_at`: no CalSoft migration adds it, and `sync/mirror.py` documents the consequence in the opposite direction, where mirroring HQ rows down raised column `"source_updated_at"` ... does not exist.

So the key is never present in an uploaded payload. `_LWW_COLUMN` in `clean_data` is therefore always `False` on the client upload path, `lww_guarded` stays `False` and the statement reduces to an unconditional `DO UPDATE SET`, **last writer to arrive wins, regardless of when the edit was made**. A stale field row uploaded after a newer HQ change overwrites it.

The guard engages only where the column is supplied explicitly: HQ-internal writes, and the two test suites, which inject `source_updated_at` into the payload by hand. That is why the tests pass while the protection is inert on the path it exists to protect.

This is the most consequential finding in this document, because the protection is documented and tested and therefore trusted.

4.2 Finding H-4, A payload omitting the version column wins unconditionally

Independent of H-3: the clause contains `OR EXCLUDED.source_updated_at IS NULL`. Where the guard *is* active, any writer that omits the column or sends it null bypasses LWW and overwrites a newer row. The safe default for a missing version is to lose, not to win. `hq_row.source_updated_at IS NULL` has the same shape on the stored side, though that one is defensible as a migration accommodation.

4.3 Finding H-5, `is_conflict` does not compare data

`mirror.RecordDifference.is_conflict` returns true whenever a row exists on both sides and both carry timestamps, the payloads are never compared. Two byte-identical rows are therefore a "conflict". `_content_hash` exists in the same module and is not used for this test. The effect is noise rather than incorrectness, but it inflates conflict counts and makes the real ones harder to see.

4.4 Finding H-6, An uploaded null can erase an allocated certificate number

H-3 has a specific and serious consequence for certificate numbers, arrived at by combining three behaviours that are each individually reasonable.

1. Offline approval sets the local `certificate_number` to `NULL` (`status approved_pending_certificate`, a `PendingCertificate` queued)
2. HQ fulfils the request and allocates `BNH-NNNN` on the HQ row
3. The client uploads its session row. `to_jsonb(t.*)` includes every local column, so the payload carries `certificate_number: null`
4. Nulls now propagate, `tests/test_null_propagation.py` exists because they used to be dropped, and that was fixed
5. The LWW guard is inert on this path (H-3), so nothing compares edit times
6. `ON CONFLICT DO UPDATE SET certificate_number = EXCLUDED.certificate_number` writes `NULL` over HQ's allocated number

Neither `upload_processing.py`, `record_client_upload.py` nor `apply_change_status_routes.py` mentions `certificate_number` at all, so there is no field-specific protection at HQ. `tests/test_cert_delivery.py` covers the delivery direction (HQ to device), not this one.

What follows from an erased number. `sync/sync_agent_1.py` runs a recovery loop that finds locally-approved sessions with no number and pulls from HQ, good engineering, and it repairs the ordinary case where the client simply has not received the number yet. But it pulls *from* HQ, and in this scenario HQ is where the number was erased. HQ's certificate service then sees an approved session with no number and allocates a fresh one,

gap-reuse first. The session ends up carrying a different number from the one originally allocated and if the first was already printed, the outcome is H-1 reached by another route.

Timing, stated precisely. This requires the client to upload its session row after HQ allocates and before the client applies the number locally. The download path (`sync_agent_4.py`) applies incoming numbers and reconciles pending certificates, so the window is a race rather than a certainty. The window's width depends on poll intervals and backlog, and a sync backlog widens it, which is what `tests/test_sync_backlog_alert.py` and the most recent HQ commit are concerned with. I have not reproduced it; the mechanism is established from the code, and closing H-3 closes this too.

5. Findings summary

Status. H-2 is fixed: local certificate-number allocation has been removed from the field application, so HQ is the sole allocator and the collision this section describes can no longer occur. The remaining items are unchanged and all land in the HQ repository or the sync client.

#	Severity	Finding	State
H-1	High	Conflict repair reassigns the certificate number of an already-issued document, with no superseded record	Open
H-2	High	Two reachable allocation algorithms on two databases; violates the server-only rule and causes the collisions H-1 repairs	Fixed
H-3	High	The LWW guard cannot engage on client uploads, <code>to_jsonb(t.*)</code> cannot emit a column the local table lacks. Confirmed	Open
H-6	High	An uploaded null can erase an allocated certificate number at HQ, which is then reallocated gap-reuse-first	Open
H-4	Medium	Where the guard is active, a payload omitting the version column bypasses it and wins	Open
H-5	Low	<code>is_conflict</code> ignores payload contents; identical rows count as conflicts	Open

These group into two pieces of work:

- **H-2 -> H-1.** Fixing allocation at source (server-only, per the stated rule) removes the collisions, which makes the conflict guard a dormant safety net rather than a working part of issuance.
- **H-3 -> H-6 -> H-4.** Giving the client a `source_updated_at` column and populating it on write makes the guard live, which closes the null-overwrite path as a side effect. Inverting the `EXCLUDED ... IS NULL` default then removes the remaining bypass.

Neither is large. H-3 is a migration plus a write-path change; H-2 is deleting a code path the stated rule already forbids.

6. What this changes in the plan

`04_PLAN_TO_9_OF_10.pdf` Phase 4 already spans both repositories and already covers signing plus numbering authority. Three amendments:

1. **Item 4.2 has an answer now.** The stated rule settles it: HQ allocates, or the online branch queues like the offline one. The reserved-range option in the plan is unnecessary, it solves a problem the rule dissolves.

2. **Add: no number bound to an issued document is ever reassigned.** The guard must void and reissue with a superseded record, or the collision must be removed at source. This is H-1 and belongs in Phase 4.
3. **H-3 is now confirmed, so Phase 2 gains a fix, not just a test.** The client needs `source_updated_at` on its synced tables, populated on write, so the guard can engage. The test that proves it, upload a stale row, assert HQ's newer value survives, belongs in Phase 2 alongside it. A guard that cannot fire is worse than no guard, because it is trusted.
4. **H-6 belongs in Phase 4.** Until the guard is live, HQ should refuse to overwrite a non-null `certificate_number` with a null one. That is a narrow, field-specific guard and it protects the artefact the whole phase is about.

None of this moves the Phase 4 score target. It makes the phase's scope accurate: it was written as "sign the certificate", and it is really "establish what a certificate number is, who may allocate one, and prove that records arrive intact."

7. Not audited

Stated so the boundary is visible rather than assumed: `og_server_core.py` (1,966 lines), the Redis queue and cache layers (2,068), `soft_delete_handler.py` on both sides beyond confirming it flags rather than deletes, `dependency_manager`, `conflict_quarantine`, `drift_reconciler` and `state_manager`. The upload payload path through `sync_agent_1` to `_4`, `smart_delete` and `upload_processing._upsert_entity` **was** traced, because H-3 and H-6 depend on it. This audit followed the certificate and record-integrity paths only.

Static analysis of both repositories. No code was executed and no code was modified. H-3 was traced end to end and is confirmed; H-6 follows from it and from three verified behaviours, with the timing caveat stated in section 4.4.

